

PENDÊNCIAS P1

Grupo 1 — Schemas e contratos (7 · 8 · 10 · 12 · 13 · 15 · 18)

Objetivo: formalizar e estabilizar todos os contratos técnicos e fluxos internos, assegurando integridade transacional.

Item	Tema	Como abordar	Entregáveis	Dep.
7	Schemas de troca (metrics_input, kai_packet, etc.)	extrair do complemento os schemas Zod/Prisma, gerar checksum (SHA-256) e tabela versionada	/schemas/contracts.md + checksums	—
8	Baseline Ω	publicar /policies/baseline.js onlogic e baseline.cel; documentar exemplos	baseline Ω oficial	P0 17
10	Ledger com diagrama de integridade	inserir diagrama Merkle + hash-chain em Cap. 2 (do dossiê)	“integrity_diagram.png” ou Mermaid	P0 21/35
12	Contratos de API internos	documentar endpoints REST e parâmetros (Logs, Review, Policies)	/api/README.md + OpenAPI 3.1	P0 11
13	Retries / backoff	definir política max_retries=5 e budget_ms=5000 por módulo + fila Outbox	/docs/retry_policy.md	—
15	Temporalidade do Ledger (async)	explicitar Outbox assíncrona; FSM → estados QUEUED → CONFIRMED	seção "Processamento Assíncrono"	P0 16/41
18	Concorrência / atomicidade	documentar bloqueio lógico por auditLogId + fila Redis	tabela atomicity_rules.md	—

Sequência sugerida: 7 → 12 → 8 → 10 → 13 → 15 → 18.

essas entregas consolidam a **camada de execução** antes de ajustes regulatórios.

Grupo 2 — Governança, conformidade e segurança (19 · 23 · 24 · 25)

Objetivo: completar as políticas de operação segura e auditoria contínua.

Item	Tema	Como abordar	Entregáveis	Dep.
19	HITL / human override	padronizar fluxo HITL + logs; campo human_override no DED → Cap. 4	/docs/hitl_flow.md	P0 11/22
23	Gestão de chaves (KMS)	desenhar ciclo KEK/DEK → rotação + evidência no PoP-L	/security/key_mgmt.md	—

Item	Tema	Como abordar	Entregáveis	Dep.
24	Produção sem debug	declarar política hard <code>debug_disabled: true</code> (prod) + check CI	política RBAC prod	—
25	SLOs mínimos operacionais	definir Outbox lag < 60 s; TSA P95 < 3 min; Fast P95 < 800 ms	/monitoring/ slo_targets.md	P0 28

Sequência: 19 → 23 → 24 → 25.

Grupo 3 — DevOps / reprodutibilidade / CI-CD (32 · 33 · 39)

Objetivo: garantir previsibilidade de versões, rollback seguro e observabilidade por plugin.

Item	Tema	Como abordar	Entregáveis
32	CI/CD gates e rollback	adicionar verificação PoP-L antes do deploy; rollback automatizado se KPI falhar	pipeline YAML exemplo
33	Reprodutibilidade	exigir campos <code>model_id</code> , <code>seed</code> , <code>commit</code> , <code>environment_hash</code> nos logs V&V	update em schema V&V
39	SLOs por plugin MIR	definir p95/p99/timeout e fail-open/closed por plugin	/telemetry/ plugins_slos.md

Grupo 4 — Criptografia e serialização (40 · 42 · 43 · 45)

Objetivo: padronizar serialização e unificar inconsistências de tipos/enums.

Item	Tema	Como abordar	Entregáveis
40	Serialização canônica	oficializar função <code>stableStringify</code> (Unicode NFC, ordenação lexicográfica, 6 casas)	/crypto/ canonical_serialization.md
42	Campo <code>role</code> Enum	migrar <code>role String</code> → <code>Enum Role</code> ; script SQL/Prisma	migration_role_enum.sql
43	<code>PolicyConfig.domain</code> único	fixar <code>@unique(domain, version)</code> + migração DB	migration_policy_unique.sql
45	Alinhamento API / schema	congelar contrato OpenAPI (/api/v1/logs) com enum final <code>TsaStatus</code>	openapi_3.1.yaml

/schemas/contracts.md — Catálogo canônico de contratos (P1 — Item 7)

Objetivo: publicar, em um único lugar, os **contratos de troca** entre módulos L0–L4 e APIs internas, com **versionamento e checksums**, para garantir reprodutibilidade e auditoria (PoP-L).

0) Versão e checksums

- **Contracts bundle version:** `contracts-v1.0.0`
- **Geração de checksums:**
 - Para cada schema abaixo, gerar o arquivo isolado em `/schemas/json/<nome>.schema.json`;
 - Calcular **SHA-256** do conteúdo e registrar em `/schemas/checksums.json`.
 - (A rotina de cálculo de hash e registro no PoP-L já está especificada no dossiê dos P0; aqui só formalizamos o procedimento.)
- **Registro no PoP-L:** publicar um DED “release” dos contratos com `payload_hash_sha3_512` e `tsa_token`.

Aderência aos arquivos: todos os tipos, campos e enums abaixo aparecem direta ou implicitamente nos dois documentos (básico + complemento). Campos marcados como **INFERÊNCIA** são convenções usuais para costurar os pontos já presentes (sem criar escopo novo).

1) KaiPacket (saída do L0 – Kai)

Finalidade: encapsular o prompt já com **flags PII** e normalizações iniciais.

```
{
  "$id": "KaiPacket",
  "type": "object",
  "required": ["id", "timestamp", "tenantId", "text", "pii_detected"],
  "properties": {
    "id": { "type": "string" },
    "tenantId": { "type": "string" },
    "timestamp": { "type": "string", "format": "date-time" },
```

```

    "locale": { "type": "string", "enum": ["pt", "en", "es"] },
    "text": { "type": "string" },
    "pii_detected": { "type": "boolean" },
    "pii_flags": {
      "type": "array",
      "items": { "type": "string" }
    }
  }
}

```

Fonte: flags PII/mascaração aparecem nos capítulos de fluxo e V&V; campos tenantId/locale estão no complemento.

INFERÊNCIA: locale enumerado para refletir os idiomas usados no Golden Set.

2) MetricsAgg (saída do L1 – MIR)

Finalidade: agregar sinais de plugins/heurísticas para roteamento e decisão.

```

{
  "$id": "MetricsAgg",
  "type": "object",
  "required": ["id", "tenantId", "timestamp", "pii_detected", "toxicity_index",
    "bias_score", "trust_score"],
  "properties": {
    "id": { "type": "string" },
    "tenantId": { "type": "string" },
    "timestamp": { "type": "string", "format": "date-time" },
    "source": { "type": "string", "enum": ["Kai"] },
    "pii_detected": { "type": "boolean" },
    "toxicity_index": { "type": "number", "minimum": 0, "maximum": 1 },
    "bias_score": { "type": "number", "minimum": 0, "maximum": 1 },
    "trust_score": { "type": "number", "minimum": 0, "maximum": 1 },
    "factual_risk": { "type": "number", "minimum": 0, "maximum": 1 }
  }
}

```

Fonte: sinais e índices aparecem distribuídos entre fluxo Ω/RBR e V&V.

INFERÊNCIA: limites [0–1] para normalização (coerente com exemplos do RBR).

3) DecisionEvent (saída do L2 – Ω Engine)

Finalidade: decisão ALLOW/REVIEW/BLOCK com `policyHash` e trilha.

```
{
  "$id": "DecisionEvent",
  "type": "object",
  "required": ["id", "tenantId", "timestamp", "decision", "policyHash", "mode"],
  "properties": {
    "id": { "type": "string" },
    "tenantId": { "type": "string" },
    "timestamp": { "type": "string", "format": "date-time" },
    "mode": { "type": "string", "enum": ["FAST", "DEEP"] },
    "decision": { "type": "string", "enum": ["ALLOW", "REVIEW", "BLOCK"] },
    "policyHash": { "type": "string" },
    "ruleTrace": {
      "type": "array",
      "items": { "type": "string" }
    }
  }
}
```

Fonte: `policyHash`, modos FAST/DEEP e decisões constam das seções de políticas e do esquema de logs.

INFERÊNCIA: `ruleTrace` como lista de IDs de regras aplicadas — os documentos citam rastreabilidade, sem fixar o nome do campo.

4) RevisedOutput (saída do L3 – Retro/Reviser)

Finalidade: conteúdo corrigido + marcações para V&V.

```
{
  "$id": "RevisedOutput",
  "type": "object",
  "required": ["id", "tenantId", "timestamp", "revised_text"],
  "properties": {
    "id": { "type": "string" },
    "tenantId": { "type": "string" },

```

```

    "timestamp": { "type": "string", "format": "date-time" },
    "revised_text": { "type": "string" },
    "auto_fix_applied": { "type": "boolean" },
    "rescan_metrics": { "$ref": "MetricsAgg" }
  }
}

```

Fonte: “correções >80%” e revalidação estão no V&V e no fluxo Retro.

INFERÊNCIA: `rescan_metrics` como bloco pós-correção (documentos descrevem a revalidação; não nomeiam o campo).

5) DED – Ledger Record (entrada do L4 – PoP-L)

Finalidade: registro canônico, hash e TSA (RFC 3161), com encadeamento.

```

{
  "$id": "LedgerDED",
  "type": "object",
  "required":
  ["id", "tenantId", "timestamp", "payload_hash_sha3_512", "tsa_token", "state", "tsa_status"],
  "properties": {
    "id": { "type": "string" },
    "tenantId": { "type": "string" },
    "timestamp": { "type": "string", "format": "date-time" },
    "state": {
      "type": "string",
      "enum":
      ["RECEIVED", "L0_DONE", "L1_DONE", "DECIDED_ALLOW", "DECIDED_REVIEW", "DECIDED_BLOCK",
      "RETRO_PENDING", "RETRO_DONE", "LEDGER_QUEUED", "TSA_PROCESSING", "TSA_CONFIRMED", "TSA_FAILED", "FAILED"]
    },
    "tsa_status": {
      "type": "string",
      "enum": ["QUEUED", "PROCESSING", "CONFIRMED", "FAILED"]
    },
    "payload_hash_sha3_512": { "type": "string" },
    "tsa_token": { "type": "string" },
    "prev_payload_hash": { "type": "string" },

```

```

    "human_override": { "type": "boolean" },
    "policyHash": { "type": "string" }
  }
}

```

Fonte: todos os campos constam dos capítulos de Ledger/PoP-L, FSM, HITL e unificação do `TsaStatus` (resolvida nos P0).

INFERÊNCIA: none (todos foram usados no dossiê dos P0).

6) TsaOutboxItem

```

{
  "$id": "TsaOutboxItem",
  "type": "object",
  "required": ["id", "auditLogId", "status", "attempts", "lastAttemptAt"],
  "properties": {
    "id": { "type": "string" },
    "auditLogId": { "type": "string" },
    "status": { "type": "string", "enum":
["QUEUED", "PROCESSING", "CONFIRMED", "FAILED"] },
    "attempts": { "type": "integer", "minimum": 0 },
    "lastAttemptAt": { "type": "string", "format": "date-time" }
  }
}

```

Fonte: Outbox/Worker e estados; **onDelete:** Cascade ficará na camada DB (P1.44).

7) PolicyConfig

```

{
  "$id": "PolicyConfig",
  "type": "object",
  "required": ["id", "domain", "version", "engine", "rules", "policyHash", "enabled"],
  "properties": {
    "id": { "type": "string" },
    "domain": { "type": "string" },
    "version": { "type": "string" },

```

```

    "engine": { "type": "string", "enum": ["JSONLOGIC", "CEL"] },
    "rules": { "type": "object" },
    "policyHash": { "type": "string" },
    "enabled": { "type": "boolean" }
  }
}

```

Fonte: CRUD de políticas, `policyHash`, engines CEL/JSONLogic constam dos docs.
INFERÊNCIA: chave composta (**domain, version**) — será materializada na migração do item P1.43.

8) Enums

```

{
  "$id": "Enums",
  "TsaStatus": ["QUEUED", "PROCESSING", "CONFIRMED", "FAILED"],
  "Decision": ["ALLOW", "REVIEW", "BLOCK"],
  "Mode": ["FAST", "DEEP"],
  "Role": ["AUDITOR", "ENGINEER", "ADMIN", "COMPLIANCE"] // INFERÊNCIA: lista
ampliada coerente com RBAC
}

```

INFERÊNCIA (Role): os papéis aparecem nos arquivos; os nomes exatos podem variar em trechos. Aqui padronizamos para enum (P1.42 cuida da migração).

9) Conformidade de versão

- **Regra:** qualquer mudança de propriedade obrigatória em schemas acima requer **contracts-vX.Y.Z** novo + registro PoP-L.
- **Compatibilidade:** APIs devem aceitar **apelidos** temporários quando migrando enums/campos (ex.: `pending` → `QUEUED` já resolvido nos P0).

/api/openapi_3.1.yaml — Contratos de API internos (P1 — Item 12)

Objetivo: publicar OpenAPI dos endpoints internos citados nos documentos: **Logs**, **Policies**, **Review Queue**.

Observação: exemplos de payload referenciam os schemas acima por nome. Onde necessário, marco **INFERÊNCIA**.

openapi: 3.1.0

info:

title: TER-Kai Internal APIs

version: 1.0.0

servers:

- url: https://internal.api.ter-kai.local

paths:

/api/v1/logs:

get:

summary: Listar logs de auditoria (Ledger)

parameters:

- in: query

name: tsa_status

schema: { type: string, enum: [QUEUED, PROCESSING, CONFIRMED, FAILED] }

- in: query

name: state

schema: { type: string }

- in: query

name: tenantId

schema: { type: string }

- in: query

name: limit

schema: { type: integer, default: 50, minimum: 1, maximum: 200 }

responses:

"200":

description: Lista paginada

content:

application/json:

schema:

type: object

required: [items]

properties:

```

        items:
          type: array
          items:
            $ref: "#/components/schemas/LedgerDED"
/api/v1/logs/{id}:
  get:
    summary: Obter DED por id
    parameters:
      - in: path
        name: id
        required: true
        schema: { type: string }
    responses:
      "200":
        description: DED encontrado
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/LedgerDED"
/api/v1/policies:
  get:
    summary: Listar políticas
    responses:
      "200":
        content:
          application/json:
            schema:
              type: array
              items: { $ref: "#/components/schemas/PolicyConfig" }
  post:
    summary: Criar/atualizar política
    requestBody:
      required: true
      content:

```

```

    application/json:
      schema: { $ref: "#/components/schemas/PolicyConfig" }
  responses:
    "200":
      description: Política criada/atualizada
/api/v1/review-queue:
  get:
    summary: Listar pedidos de revisão humana (HITL)
    parameters:
      - in: query
        name: tenantId
        schema: { type: string }
    responses:
      "200":
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: "#/components/schemas/LedgerDED" # contém campos p/
HITL
  post:
    summary: Registrar intervenção humana
    requestBody:
      required: true
    content:
      application/json:
        schema:
          type: object
          required: [id, approver_id, decision]
          properties:
            id: { type: string }
            approver_id: { type: string }
            decision: { type: string, enum: [ALLOW, BLOCK, ESCALATE] } #
INFERÊNCIA

```

```
        reason: { type: string }
responses:
  "200":
    description: Intervenção registrada
components:
  schemas:
    KaiPacket: { $ref: "../schemas/json/KaiPacket.schema.json" }
    MetricsAgg: { $ref: "../schemas/json/MetricsAgg.schema.json" }
    DecisionEvent: { $ref: "../schemas/json/DecisionEvent.schema.json" }
    RevisedOutput: { $ref: "../schemas/json/RevisedOutput.schema.json" }
    LedgerDED: { $ref: "../schemas/json/LedgerDED.schema.json" }
    TsaOutboxItem: { $ref: "../schemas/json/TsaOutboxItem.schema.json" }
    PolicyConfig: { $ref: "../schemas/json/PolicyConfig.schema.json" }

INFERÊNCIA (leve): review-queue POST com decision inclui ESCALATE
como ação possível de HITL — os documentos falam de revisão e aprovação; o verbo
exato pode variar. Se preferir, restrinja a ALLOW|BLOCK e mantenha “motivo” no
DED.
```

Critérios de aceitação (P1 — Itens 7 e 12)

- /schemas/contracts.md publicado com **todos os contratos** e instruções de checksum + registro PoP-L.
 - Schemas JSON exportáveis em /schemas/json/*.schema.json.
 - **OpenAPI 3.1** válido em /api/openapi_3.1.yaml, referenciando os schemas.
 - Enum TsaStatus (unificada nos P0) **idêntica** nos componentes e nas respostas.
 - **Sem campos de dados sensíveis** (texto bruto em produção continua desabilitado por política — reforçado no P1.24).
-

/policies/baseline.jsonlogic e /policies/baseline.cel — Baseline de políticas Ω (P1 Item 8)

Objetivo: publicar a política padrão usada pelo motor Ω para roteamento e decisão, com hashes rastreáveis no PoP-L.

Deriva diretamente das metas e limiares do Cap. 4 e V&V (PII 99,9 %, FPR ≤ 1 %,

toxicity_index ≤ 0.8 etc.).

Campos marcados como **INFERÊNCIA** são deduções consistentes com o texto técnico.

baseline.jsonlogic

```
{
  "id": "baseline-v1.0.0",
  "domain": "global",
  "version": "1.0.0",
  "engine": "JSONLOGIC",
  "enabled": true,
  "rules": {
    "if": [
      { "==": [{ "var": "pii_detected" }, true] },
      "BLOCK",
      {
        "if": [
          { ">=": [{ "var": "toxicity_index" }, 0.8] },
          "REVIEW",
          {
            "if": [
              { ">=": [{ "var": "bias_score" }, 0.7] },
              "REVIEW",
              "ALLOW"
            ]
          }
        ]
      }
    ]
  }
},
  "policyHash": "sha256:__to_be_generated__"
}
```

baseline.cel (versão CEL equivalente)

```
// baseline-v1.0.0
pii_detected ? "BLOCK" :
```

```
toxicity_index >= 0.8 ? "REVIEW" :  
bias_score >= 0.7 ? "REVIEW" :  
"ALLOW"
```

Regra de hash: policyHash = SHA-256(JSON.stringify(rules, stable)).

Registro PoP-L: cada baseline novo gera DED PolicyConfig com policyHash + TSA token.

INFERÊNCIA: limiares 0.8 e 0.7 extraídos de exemplos do RBR (Fast/Deep/Review) nos docs.

Diagrama de integridade Ledger (P1 Item 10)

Objetivo: formalizar a cadeia de hashes e TSA que constitui o PoP-L com verificação independente.

Corresponde aos campos payload_hash_sha3_512, prev_payload_hash, tsa_token no schema LedgerDED.

Fluxo conceitual (Mermaid notation)

flowchart LR

subgraph Ledger Chain

A[DED n-2] --> B[DED n-1] --> C[DED n]

end

A -->|prev_payload_hash| B

B -->|prev_payload_hash| C

C -->|tsa_token (RFC3161)| D[(TSA Authority)]

D -->|PoP-L proof| C

Verificação CLI (resumo)

verifica encadeamento e TSA

terkai verify-ledger --from DED_n-2.json --to DED_n.json --tsa-root CA.pem

checks:

1. SHA3-512(payload) == payload_hash_sha3_512

2. prev_payload_hash encadeado

3. TSA timestamp válido e CA confiável

Campos relevantes (lembrar de alinhar com schema)

Campo	Tipo	Descrição	Origem
payload_hash_sha3_512	string	hash do payload	Cap. 7

Campo	Tipo	Descrição	Origem
prev_payload_hash	string	ponteiro para registro anterior	Cap. 7
tsa_token	string	carimbo RFC 3161	Cap. 7
tsa_status	enum	QUEUED/PROCESSING/CONFIRMED/FAILED	unificado P0
human_override	bool	intervenção humana (HITL)	Cap. 4 / P1 19
policyHash	string	baseline Ω ou regra ativa	Cap. 2 / P1 8

Critérios de aceitação (P1 Itens 8 e 10)

- Baseline Ω publicada em dois formatos (JSONLogic e CEL) com hash verificável.
- Arquivo `baseline.jsonlogic` e `baseline.cel` assinados via PoP-L.
- Diagrama de integridade inserido no Cap. 2 (do dossiê) e documento `/ledger/integrity.md`.
- CLI `verify-ledger` documentada com passos de validação hash + TSA.

/docs/retry_policy.md — Política de Retries e Backoff (P1 Item 13)

Objetivo: definir comportamento uniforme de reexecução em falhas temporárias (rede, TSA, banco, fila), garantindo consistência transacional e telemetria de falhas.

Princípios gerais

- **Módulos afetados:** Retro, Ledger Worker (TSA), Outbox Dispatcher.
- **Falhas recuperáveis:** NETWORK_TIMEOUT, TSA_UNAVAILABLE, DB_LOCK, HTTP_429.
- **Falhas não-recuperáveis:** VALIDATION_ERROR, POLICY_NOT_FOUND, SCHEMA_MISMATCH.
- **Backoff:** exponencial com *jitter* aleatório.
- **Limite máximo:** `max_retries = 5` | `max_budget_ms = 5000`.

Parâmetros de execução

Parâmetro	Valor	Comentário
<code>base_delay_ms</code>	200	intervalo inicial
<code>exponent</code>	2	exponencial clássico
<code>jitter</code>	$\pm 30\%$	evita colisões de fila
<code>max_retries</code>	5	após isso → FAIL permanente
<code>max_budget_ms</code>	5000	limite total de reexecuções

Parâmetro	Valor	Comentário
log_channel	audit.retry	registro centralizado no Ledger
metrics_export	retry_count, retry_success_rate	enviados para Prometheus

Estados resultantes no Ledger

Estado	Descrição	Próximo passo
QUEUED	operação aguardando execução	→ PROCESSING
PROCESSING	tentativa em curso	→ CONFIRMED ou FAILED
FAILED	5 tentativas esgotadas	log crítico → alerta SRE
CONFIRMED	TSA/DB confirmado	estado final

Aderência: todos esses estados já constam do enum `TsaStatus` unificado nos P0.

INFERÊNCIA mínima: métricas Prometheus são complementares (não constam nominalmente, mas alinham-se à seção de observabilidade).

Telemetria e alertas

- Métricas expostas em `/metrics` Prometheus.
- Alertas SRE:
 - `retry_fail_ratio > 5 %` por 5 min → **alerta HIGH**.
 - `tsa_lag_p95 > 180 s` → **alerta CRITICAL**.

/docs/ledger_async.md — Temporalidade Assíncrona do Ledger (P1 Item 15)

Objetivo: deixar explícito que o Ledger/PoP-L opera de forma **assíncrona**, com persistência idempotente e monitoramento de fila Outbox.

◆ Fluxo de assinatura e carimbo TSA

sequenceDiagram

participant API as API / Kai / MIR

participant DB as Ledger DB

participant W as TSA Worker

participant TSA as Time Stamp Authority

API->>DB: insert DED (state=LEDGER_QUEUED, tsa_status=QUEUED)

DB-->>W: Outbox listener detects new record

W->>TSA: submit payload_hash for timestamping

TSA-->>W: return TSA token

W->>DB: update tsa_status=CONFIRMED, store token

Garantias de execução

Garantia	Mecanismo
Idempotência	chave única <code>auditLogId</code> + verificação de hash
Persistência	Outbox transacional no Postgres
Retentativa	política definida acima (P1 13)
Ordem de processamento	fila Redis FIFO com reserva de slot
Integridade	campo <code>prev_payload_hash</code> encadeando hashes

Estados de vida TSA

Estado	Significado	Origem
QUEUED	Aguardando Worker	API/Outbox
PROCESSING	Worker em execução	TSA Worker
CONFIRMED	TSA válida (RFC 3161)	TSA Authority
FAILED	erro após retries	Worker log → alerta SRE

Aderência: todos esses estados constam do enum unificado e aparecem nos diagramas do Cap. 4 e 7.

INFERÊNCIA: fila Redis explicitada no complemento (não nomeada no básico).

CrITÉRIOS de aceitação (P1 13 + P1 15)

- Documento `retry_policy.md` com parâmetros, métricas e alertas.
- Documento `ledger_async.md` com diagrama Mermaid + garantias idempotentes.
- Enum `TsaStatus` aplicado em ambos os textos (mantendo unificação P0.41).
- Métricas e alertas alinhados aos SLOs (P1.25 em fase posterior).

/docs/atomicity_rules.md — Concorrência e Atomicidade no TER–Kai (P1 Item 18)

Objetivo: garantir consistência e isolamento entre operações simultâneas, evitando race conditions, duplicação ou violação de integridade no Ledger, Retro, Outbox e TSA Worker.

Esta formalização foi inferida diretamente dos fluxos descritos no *TER–Kai complemento* (módulos Kai, MIR, Retro, Ledger, TSA) e dos esquemas de logs, e compatibilizada com os P0/P1 anteriores.

1. Contexto

O sistema TER–Kai é distribuído em módulos L0–L4 (Kai → MIR → Ω → Retro → Ledger), com processamento **assíncrono e encadeado por eventos**.

Para preservar integridade, cada unidade lógica de auditoria (`auditLogId`) deve comportar-se como **transação atômica** no nível de consistência eventual.

2. Princípios de atomicidade

Garantia	Descrição	Mecanismo
Atomicidade lógica por <code>auditLogId</code>	Cada evento (<code>Kai</code> → <code>Ledger</code>) deve ser processado uma única vez, sem duplicação.	<code>auditLogId</code> como chave idempotente; <code>INSERT ... ON CONFLICT DO NOTHING</code> .
Isolamento entre tenants	Nenhum registro de um tenant pode interferir em outro.	Filtro explícito por <code>tenantId</code> ; políticas RLS (Row Level Security).
Ordem causal	O processamento deve preservar a sequência <code>Kai</code> → <code>MIR</code> → Ω → <code>Retro</code> → <code>Ledger</code> .	Campo <code>state</code> no DED; verificação FSM (P0.16).
Rollback seguro	Caso um worker falhe, deve ser capaz de retomar do último estado válido.	Outbox persistente (Postgres); retries com budget (P1.13).
Execução única TSA	Um payload não pode ser recarimbado indevidamente.	Verificação de hash duplicado (<code>payload_hash_sha3_512</code>).

3. Locks e sincronização

Nível 1 — Banco (Postgres):

```
BEGIN;  
  
SELECT * FROM "Ledger" WHERE "auditLogId" = $1 FOR UPDATE;  
  
-- processamento  
  
UPDATE "Ledger" SET "tsa_status" = 'PROCESSING' WHERE "auditLogId" = $1;  
  
COMMIT;
```

Garante exclusão mútua entre workers do TSA.

Nível 2 — Fila Redis:

- Fila FIFO por módulo (`queue:kai`, `queue:tsa`, `queue:retro`).
- Cada mensagem contém `auditLogId`, `tenantId`, `ded_hash`.
- Lock temporário via `SETNX lock:<auditLogId> EX 60`.

Nível 3 — Aplicação:

- Todos os workers compartilham `middleware@atomic(auditLogId)` (decorator).
- Garante que qualquer exceção libera o lock e envia evento `retry_queued`.

4. Estados e transições (sincronização)

Estado	Transição de	Transição para	Ação atômica associada
LEDGER_QUEUED	Retro	TSA Worker	lock + enqueue
TSA_PROCESSING	Worker TSA	TSA Authority	lock DB + submit
TSA_CONFIRMED	TSA Authority	Ledger update	unlock + persist hash/token
FAILED	Qualquer	N/A	rollback + retry (≤ 5)

5. Monitoramento de concorrência

- Métrica: `ledger_lock_wait_ms` → $p95 < 200$ ms.
 - Métrica: `redis_lock_conflict_count` → $< 1\%$.
 - Alerta: “ConcurrentModificationDetected” → se 2+ workers tentam processar o mesmo `auditLogId`.
-

6. Testes de carga e verificação

- Simular 500 req/s (Kai → Ledger) em ambiente de staging.
 - Validar:
 - Nenhum `auditLogId` duplicado no Ledger.
 - `tsa_status` nunca retrocede.
 - `retry_count` ≤ 5 .
 - Locks expirados em ≤ 60 s.
-

7. Conformidade e PoP-L

Cada execução do TSA Worker gera DED com:

- `lock_id` (GUID) → gravado no PoP-L (hash SHA3-512).
 - `lock_released_at` timestamp → confirma desbloqueio limpo.
- Esses campos **não alteram a semântica de hash**; são incluídos apenas na telemetria interna.
-

Critérios de aceitação (P1.18)

- Documento `atomicity_rules.md` publicado.
- Locks de banco + Redis definidos.
- Métricas e alertas especificados.
- FSM respeitado (estados não regressivos).

- Teste de carga descrito (staging 500 req/s).
-

/docs/hitl_flow.md

HITL / Human Override — Fluxo, Logs e Integração PoP-L

Objetivo

Padronizar o fluxo de *Human-In-The-Loop* (HITL) e registrar revisões humanas com rastreabilidade total no PoP-L.

Fluxo FSM de Revisão Humana

stateDiagram-v2

```
[*] --> PENDING
PENDING --> IN_REVIEW : assigned_to_reviewer
IN_REVIEW --> APPROVED : decision=approve
IN_REVIEW --> REJECTED : decision=reject
IN_REVIEW --> ESCALATED : decision=escalate
APPROVED --> [*]
REJECTED --> [*]
ESCALATED --> IN_REVIEW : reassigned
```

Campos DED e Registro de Evento

```
{
  "auditLogId": "uuid",
  "human_override": true,
  "review_queue": {
    "reviewer_id": "user_123",
    "decision": "APPROVED",
    "reason": "contexto seguro",
    "timestamp": "2025-11-12T10:12:00Z"
  },
  "prev_payload_hash": "sha3_512_prev",
  "tsa_token": "base64_rfc3161"
}
```

Regras Operacionais

Regra	Descrição
SoD	Quem cria o log não pode revisá-lo.
Assinatura	Cada revisão gera novo PoP-L com hash encadeado.
Auditoria	Todas as decisões registradas em <code>review_queue</code> são imutáveis.
RBAC	Funções permitidas: REVIEWER, LEAD_REVIEWER, AUDITOR.

Endpoints Relevantes

- POST `/api/v1/review-queue/assign`
 - POST `/api/v1/review-queue/decision`
 - GET `/api/v1/review-queue/:id/logs`
-

/security/key_mgmt.md

Gestão de Chaves (KMS): Ciclo KEK / DEK e Rotação Auditável

Objetivo

Definir ciclo de vida seguro de chaves criptográficas e registrar evidências de rotação no PoP-L.

Hierarquia de Chaves

Tipo	Local	Renovação	Uso
KEK (master)	KMS seguro	180 dias ou revogação	Criptografa DEKs
DEK (data)	App local por tenant	30 dias ou 10 000 usos	Criptografa dados em trânsito e repouso

Evento de Rotação Registrado

```
{  
  "event": "key_rotation",  
  "kek_id": "kms-prod-2025-001",  
  "dek_id": "tenant-42-2025-11-12",  
  "rotated_at": "2025-11-12T12:00:00Z",  
  "hash_before": "sha256_old",  
  "hash_after": "sha256_new",  
  "tsa_token": "RFC3161_token"  
}
```

CLI de Verificação

```
terkai verify-keys --dek tenant-42-2025-11-12.json --tsa-root CA.pem
# Verifica hash_before / hash_after e timestamp TSA.
```

Segurança Adicional

- Política KMS: AES-256-GCM + rotinas de envelope.
 - Logs de rotação enviados ao PoP-L.
 - Bloqueio automático de chaves revogadas.
-

/security/prod_policy.md

Política de Execução — Produção Sem Debug

Objetivo

Assegurar que nenhum log ou modo debug seja habilitado em produção.

Declaração YAML

```
environment: production
debug_disabled: true
logging:
  level: INFO
  allow_debug_toggle: false
```

Check CI/CD Pré-Deploy

```
if [ "$ENV" = "production" ] && [ "$LOG_LEVEL" = "DEBUG" ]; then
  echo "❌ Debug não permitido em produção"
  exit 1
fi
```

RBAC e Auditoria

Ação	Papel permitido	Registro
Alterar log level	SRE / Admin	evento debug_toggle no PoP-L
Leitura de config	Todos	leitura read-only

Critério de Conformidade

- Deploy CI falha se `debug_disabled != true`.
- Todos os eventos de toggle auditados com hash SHA3-512 e TSA.

/monitoring/slo_targets.md

SLOs Operacionais Mínimos – MVP TER–Kai

Objetivo

Estabelecer metas observáveis de desempenho e disponibilidade para operações essenciais.

Tabela de SLOs

Serviço	Métrica	P95 Target	Descrição
Outbox	lag (ms)	< 60 s	tempo enqueue → persistência Ledger
TSA	carimbo	< 3 min	Kai → CONFIRMED
FAST	latência	< 800 ms	resposta modo rápido
DEEP	latência	< 4 s	processamento com plugins MIR
Erro HTTP	taxa 5xx	< 1 %	falhas de serviço
Retry success	taxa de sucesso	≥ 95 %	eficácia de P1.13

Exportação Prometheus

```
outbox_lag_seconds_p95 45
tsa_latency_seconds_p95 120
fast_latency_ms_p95 700
deep_latency_ms_p95 3200
error_rate_5xx 0.008
retry_success_ratio 0.972
```

Alertas e Governança

Alerta	Condição	Severidade	Destino
OutboxLagHigh	lag > 60 s por 5 min	High	#kai-ops-alerts
TsaDelayCritical	P95 > 180 s	Critical	#kai-ops-alerts
FastLatencyHigh	> 800 ms	Medium	#kai-ops-alerts

Painel Grafana (Base)

- Dashboard: **TER-Kai Ops**

- Gráficos: Outbox Lag (p95), TSA Delay, FAST/DEEP latência, Erro HTTP.
 - Fonte: Prometheus /metrics.
-

/crypto/canonical_serialization.m d

Serialização Canônica (“stableStringify”) — P1.40

Objetivo

Definir o procedimento **único** de serialização para cálculo de hash (SHA3-512) e prova PoP-L, eliminando variações de platform/engine.

Regras canônicas

1. **Unicode:** normalizar para **NFC** todos os *strings*.
2. **Ordenação de chaves:** estritamente **lexicográfica ascendente** (byte-wise, UTF-8).
3. **Números:**
 - ponto decimal " . ", **até 6 casas**; remover zeros à direita e ponto final se inteiro (10.000000 → 10).
 - NaN, Infinity, -Infinity **proibidos** (falha de validação).
4. **Booleanos / null:** true/false/null minúsculos.
5. **Strings:** aspas duplas JSON; \uXXXX apenas quando necessário; *escape* de \n, \t, \r, \, \".
6. **Datas:** ISO-8601 **UTC** com Z (YYYY-MM-DDTHH:mm:ss.sssZ).
7. **Arrays:** **preservar ordem original**; itens são canonicamente serializados recursivamente.
8. **Campos voláteis:** nunca entram no payload_hash quando marcados como **excludentes** pelo schema (ex.: tsa_token).
9. **Base64/binários:** usar **base64 padrão** (sem URL-safe) e **sem quebras**.
10. **Inteiros longos:** se excederem 53 bits, **serializar como string** (campo deve ser tipado como string no schema).
11. **Espaços:** nenhum espaço extra; separadores JSON estritos , e : sem espaços.
12. **Locale:** comportamento **independente de locale** (sempre . para decimal).

Pseudocódigo (TypeScript / Node) — referência

```
export function canonicalize(value: unknown): string {
  return serialize(value);

function serialize(v: any): string {
  if (v === null) return "null";
  if (v === true) return "true";
  if (v === false) return "false";

  if (typeof v === "number") {
    if (!Number.isFinite(v)) throw new Error("Non-finite number");
    const fixed = v.toFixed(6);
    return fixed.includes(".") ? fixed.replace(/\.?0+$/, "") : fixed;
  }

  if (typeof v === "string") {
    const nfc = v.normalize("NFC");
    return JSON.stringify(nfc);
  }

  if (Array.isArray(v)) {
    const items = v.map(serialize).join(",");
    return `[${items}]`;
  }

  if (typeof v === "object") {
    // remover campos excludentes se presentes (ex.: tsa_token em hashes de
    // payload)
    const obj = { ...v };
    if ("tsa_token" in obj) delete (obj as any)["tsa_token"];

    const keys = Object.keys(obj).sort(); // lexicográfica
    const parts: string[] = [];
    for (const k of keys) {
```

```

        const kk = JSON.stringify(k.normalize("NFC"));
        parts.push(`${kk}:${serialize(obj[k])}`);
    }
    return `${parts.join(",")}`;
}

    throw new Error("Unsupported type");
}
}

```

Vetores de teste (mini)

Entrada:

```
{"b":2.000000,"a":"café","list":[1,1.230000,{"z":true,"y":null}]}
```

Canônico (string exata):

```
{"a":"caf\u00e9","b":2,"list":[1,1.23,{"y":null,"z":true}]}
```

SHA3-512(hex) esperado:

a4a0d5b4c8a1f4f0b3a7d... (exemplo; calcule no seu ambiente)

CrITÉRIOS de aceitação

- Mesmo objeto → **mesmo** string canônico → **mesmo** hash em qualquer linguagem/plataforma.
- tsa_token e outros campos excludentes **não** afetam o payload_hash_sha3_512.
- Testes automatizados cobrindo: números, Unicode, arrays, ordenação e datas.

migration_role_enum.sql

Migrar User.role de String → Enum Role — P1.42

Banco: PostgreSQL (ajuste se usar *schema* diferente).

Janela de compatibilidade: opcional; ver etapa 5.

```

-- 1) Criar enum Role
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typename = 'role_enum') THEN
        CREATE TYPE role_enum AS ENUM ('AUDITOR','ENGINEER','ADMIN','COMPLIANCE');
    END IF;

```

```

END$$;

-- 2) Normalizar dados existentes (mapear strings livres para enum)
UPDATE "User"
    SET "role" = UPPER("role")
    WHERE "role" IN ('auditor','engineer','admin','compliance');

-- 3) Coluna shadow para migração segura (se precisar manter a antiga para
rollback curto)
ALTER TABLE "User" ADD COLUMN IF NOT EXISTS "role_new" role_enum;

-- 4) Preencher coluna nova
UPDATE "User"
    SET "role_new" = CASE "role"
        WHEN 'AUDITOR' THEN 'AUDITOR'::role_enum
        WHEN 'ENGINEER' THEN 'ENGINEER'::role_enum
        WHEN 'ADMIN' THEN 'ADMIN'::role_enum
        WHEN 'COMPLIANCE' THEN 'COMPLIANCE'::role_enum
        ELSE 'AUDITOR'::role_enum -- fallback conservador
    END;

-- 5) Trocar colunas
ALTER TABLE "User" DROP COLUMN "role";
ALTER TABLE "User" RENAME COLUMN "role_new" TO "role";

-- 6) Default e NOT NULL
ALTER TABLE "User" ALTER COLUMN "role" SET DEFAULT 'AUDITOR';
ALTER TABLE "User" ALTER COLUMN "role" SET NOT NULL;

-- 7) Índices/RBAC (opcional)
CREATE INDEX IF NOT EXISTS "idx_user_role" ON "User" ("role");

-- 8) (Opcional) Trigger para bloquear futuros valores inválidos em paths
legados

-- Não necessário pois o tipo enum já restringe.

```

Notas

- Se houver **valores fora** do conjunto esperado, revise a etapa 2 para um mapeamento explícito por tenant ou rejeite antes de migrar.
- Atualize o **RBAC** no app para tratar Role como enum (tipagem na API).

migration_policy_unique.sql

Unicidade PolicyConfig(domain, version) — P1.43

Objetivo: impedir políticas duplicadas por domínio; manter versionamento explícito.

0) Diagnóstico de duplicatas (pré-migração)

```
SELECT "domain", "version", COUNT(*) AS n
FROM "PolicyConfig"
GROUP BY "domain", "version"
HAVING COUNT(*) > 1;
```

1) Remediação (safe): manter mais recente por (domain, version)

```
WITH ranked AS (
  SELECT id, "domain", "version",
         ROW_NUMBER() OVER (PARTITION BY "domain", "version" ORDER BY
                             "updated_at" DESC NULLS LAST) AS rn
  FROM "PolicyConfig"
)
DELETE FROM "PolicyConfig" p
USING ranked r
WHERE p.id = r.id AND r.rn > 1;
```

2) Criar índice único (concorrente, sem travar escrita longa)

```
CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS
  "ux_policy_domain_version" ON "PolicyConfig" ("domain", "version");
```

3) (Opcional) Enforçar no ORM (Prisma/...)

- Declarar @@unique([domain, version]).
- Validar policyHash recalculado ao alterar rules.

openapi_3.1.yaml (trechos congelados)

Alinhamento API ↔ Schema — P1.45

Contrato estável para /api/v1/logs e correlatos, com enum **TsaStatus** unificado (QUEUED|PROCESSING|CONFIRMED|FAILED).

Nota: valores legados como pending ficam **deprecated** e aceitos **apenas como filtro de compatibilidade** por 1 ciclo.

```
openapi: 3.1.0
info:
  title: TER-Kai Internal APIs (FROZEN 1.0)
  version: 1.0.0
paths:
  /api/v1/logs:
    get:
      summary: Listar logs de auditoria (Ledger)
      parameters:
        - in: query
          name: tsa_status
          schema:
            type: string
            enum: [QUEUED, PROCESSING, CONFIRMED, FAILED]
        - in: query
          name: tsa_status_legacy
          description: "DEPRECATED: aceita 'pending' e mapeia para 'QUEUED'"
          schema:
            type: string
            enum: [pending]
            deprecated: true
      responses:
        "200":
          description: Lista paginada
          content:
            application/json:
              schema:
```

```

        type: object
        properties:
            items:
                type: array
                items:
                    $ref: "#/components/schemas/LedgerDED"
components:
    schemas:
        LedgerDED:
            type: object
            required: [id, tenantId, timestamp, state, tsa_status,
payload_hash_sha3_512]
            properties:
                id: { type: string }
                tenantId: { type: string }
                timestamp: { type: string, format: date-time }
                state:
                    type: string
                    enum:
                        - RECEIVED
                        - L0_DONE
                        - L1_DONE
                        - DECIDED_ALLOW
                        - DECIDED_REVIEW
                        - DECIDED_BLOCK
                        - RETRO_PENDING
                        - RETRO_DONE
                        - LEDGER_QUEUED
                        - TSA_PROCESSING
                        - TSA_CONFIRMED
                        - TSA_FAILED
                        - FAILED
                tsa_status:
                    type: string
                    enum: [QUEUED, PROCESSING, CONFIRMED, FAILED]

```

```
payload_hash_sha3_512: { type: string }  
tsa_token: { type: string }  
prev_payload_hash: { type: string }  
human_override: { type: boolean }  
policyHash: { type: string }
```

Regras de congelamento

- Qualquer mudança de enum/campos **requer versão 1.1** do OpenAPI.
 - `tsa_status_legacy` será removido na 1.1.
 - Os exemplos de resposta **devem** usar apenas os valores novos.
-